Progetto LAM 2026: ParkMate App

Luglio 2026



Eleonora Ferrari Sara Toma

1. Introduzione

1.2 Descrizione e funzionalità

ParkMate è un'applicazione che consente la gestione delle proprie sessioni di parcheggio e dei propri veicoli, corredata di funzionalità aggiuntive quali la visualizzazione dei dati storici (mediante tre differenti formati) e il calcolo di "qualità" di un parcheggio basato sul tragitto, da quest'ultimo fino alla destinazione finale dell'utente. L'applicazione è dotata di diverse schermate con interfacce mappa (implementate tramite Google Maps SDK) che consentono una visualizzazione contestualizzata alla posizione delle proprie sessioni di parcheggio. Queste sono caratterizzate dalla posizione (che può essere quella corrente acquisita tramite il GPS, una *Saved Location* personalizzata dall'utente o una selezione manuale sulla mappa), dal tipo di parcheggio (gratis, a tariffa oraria o con biglietto fisso) e dati ad esso collegati, dalla data di inizio e, eventualmente, da una nota ed una foto scattata al momento della registrazione, insieme alle informazioni del veicolo parcheggiato. A quest'ultimo è associato un nome, un tipo (macchina, moto, bici o altro) ed una fotografia selezionata dalla galleria. Grazie all'opzionale e personalizzabile - e, dunque, non invasivo - sistema di notifiche, l'utente riceve aggiornamenti in tempo reale delle sessioni di parcheggio attive e avvisi nel caso di scadenze prossime. ParkMate è inoltre disponibile in lingua inglese (default) e in italiano. Infine, l'applicazione consente di registrare sessioni di parcheggio anche nel caso di assenza di connessione WiFi, mediante l'acquisizione delle coordinate GPS della posizione corrente (o, equivalentemente, la selezione di una *Saved Location*), dati salvati nel database e sempre disponibili.

2. Architettura del Sistema

2.1 Architettura generale

L'implementazione dell'applicazione si è basata sulle *Google Recommended Architecture Guidelines* per Android, seguendo in particolare la struttura della Clean Architecture. Più nello specifico, il codice è suddiviso negli strati *Data Layer*, contenente i dati dell'applicazione e le regole di interazione con il database, *Domain Layer*, nel quale sono state sviluppate le regole di business, use cases e le interfacce dei Repository, insieme ad eventuali funzioni utility, e *Presentation Layer*, contenente le varie schermate dell'app e gli stati ad esse associati, il tutto orchestrato dai View Model. In particolare, la parte grafica di ParkMate è stata realizzata tramite il toolkit Jetpack Compose e facendo riferimento al design pattern MVVM. Infine, sono presenti apposite cartelle per la gestione di servizi generali ed infrastrutture, quali l'amministrazione dei permessi, l'implementazione delle notifiche, l'organizzazione dei task in background tramite WorkManager. Ciascuno dei componenti sopra elencati verrà successivamente esplorato nel dettaglio, corredato da frammenti di codice e overview dell'applicazione risultante.

```
└── com.example.parkmate/
    ├── core/
    ├── di/
    ├── feature_addSession/
    ├── feature_history/
    ├── feature_home/
    ├── feature_savedLocations/
    ├── feature_parkingEffort/
    ├── feature_vehiclesManager/
    ├── feature_settings/
    ├── notifications/
    ├── permissions/
    ├── service/
    ├── ui/
    ├── workManager/
    ├── BackgroundTaskManager.kt
    ├── MainActivity.kt
    └── ParkMate.kt
```

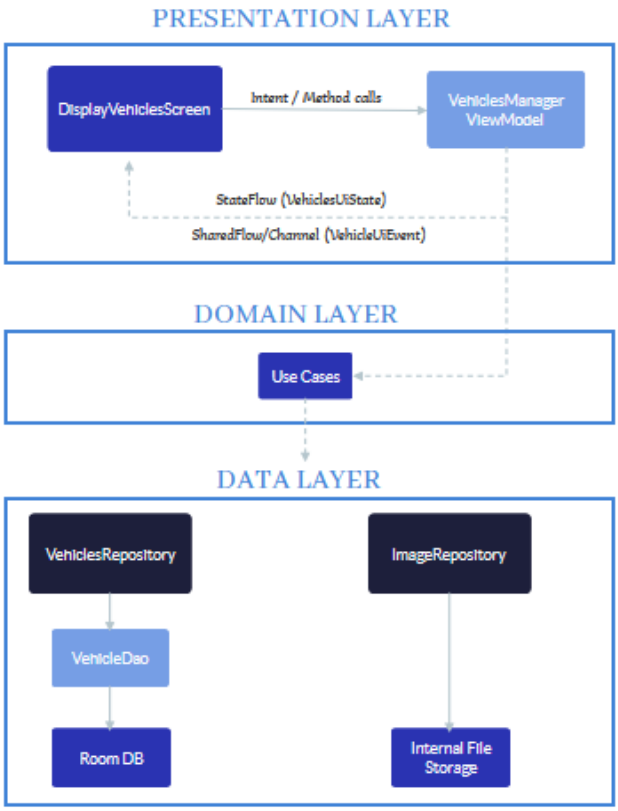
Per quanto riguarda la gestione dei dati (data e domain layer), dal momento che la maggior parte di questi viene interamente, o parzialmente, acceduta da ogni schermata, si è optato per un approccio centralizzato all'interno della cartella **`/core`**. Ad esempio, l'elenco dei veicoli salvati dall'utente viene acceduto sia da **`/feature_vehiclesManager`** (lettura/modifica), che da **`/feature_history`** (lettura).

2.2 Pattern Unidirectional Data Flow (UDF)

L'architettura dell'interfaccia utente si basa sul pattern Unidirectional Data Flow (UDF) per garantire la separazione delle responsabilità e la prevedibilità dello stato. Ogni modulo funzionale (organizzato nella cartella denominata **`/feature_nomeSchermata`**) divide nettamente le responsabilità tra UI (Screen) e ViewModel. Il flusso circolare dei dati si articola in quattro fasi:

1. Eventi: l'utente interagisce con la UI generando un evento
2. Elaborazione: il ViewModel intercetta l'evento ed esegue la logica di business (tramite gli UseCases)
3. Stato: il ViewModel aggiorna lo UI State immutabile
4. Render: La UI osserva lo stato e si aggiorna reattivamente

Segue un esempio concreto con snippet di codice per quanto riguarda l'eliminazione di una Saved Location.



1. Definizione ed emissione dell'evento: tutte le azioni utente sono modellate tramite `sealed interface`, con la UI che invia l'evento a seguito dell'interazione:

```
// Invocazione nella UI (Dropdown Menu)
DropdownMenuItem(
    text = { Text(text = stringResource(id = R.string.delete), ..)},
    onClick = {
       showMenu = false
       onEvent (SavedLocationsEvent.DeleteSavedLocation(location))
   })


// Modellazione dell'evento

sealed interface SavedLocationsEvent {
    data class DeleteSavedLocation(val location: SavedLocation):SavedLocationsEvent
}
```

2. Gestione dell'evento e aggiornamento dello Stato (ViewModel): il `ViewModel` riceve l'evento nel metodo `onEvent()`, invoca lo UseCase preposto e aggiorna lo stato visibile all'esterno tramite StateFlow immutabile:

```
@HiltViewModel
class SavedLocationsViewModel @Inject constructor(
    private val useCases: SavedLocationUseCases
) : ViewModel() {
    private val _state = MutableStateFlow(SavedLocationsState())
    val state: StateFlow<SavedLocationsState> = _state
        .onStart { loadLocations() }
        .stateIn(
            scope = viewModelScope,
            started = SharingStarted.WhileSubscribed(5000),
            initialValue = SavedLocationsState(isLoading = true)
        )

fun onEvent(event: SavedLocationsEvent){
        when(event){
              is SavedLocationsEvent.DeleteSavedLocation -> {
                viewModelScope.launch {
                    useCases.deleteSavedLocation(event.location)
                }
          }
}
```

3. Modellazione dello Stato ed ascolto nella UI: lo stato è rappresentato da una data class immutabile. La schermata Compose lo osserva in modo sicuro rispetto al ciclo di vita tramite collectAsStateWithLifecycle():

```
// Stato della Schermata
data class SavedLocationsState (
    val locations: List<SavedLocation> = emptyList(),
)
```

```
// Osservazione nella UI
 fun SavedLocationsScreen (
     viewModel: SavedLocationsViewModel = hiltViewModel(),
  ){
     val state by viewModel.state.collectAsStateWithLifecycle()
     ...
 }
```

2.3 Dependency Injection

La configurazione della Dependency Injection è centralizzata nel package **`/di`**, il quale contiene i moduli Hilt preposti a definire le istruzioni necessarie su come creare e fornire le istanze degli oggetti e componenti software condivisi nell'applicazione, quali il Database, i DAO e i Repository. Sono state usate funzioni annotate con:

- @Provides: indica a Hilt come costruire ed erogare un'istanza di un determinato tipo
- @Singleton: vincola l'istanza a essere unica per l'intero ciclo di vita dell'applicazione (ad esempio, garantendo la presenza di un'unica istanza del database Room)

```
@Module
@InstallIn(SingletonComponent::class)
object AppModule {
    @Provides
    @Singleton
    fun provideParkMateDatabase(
        @ApplicationContext context: Context
    ): ParkMateDatabase {
        val scope = CoroutineScope(Dispatchers.IO + SupervisorJob())
        lateinit var database: ParkMateDatabase

        database = Room.databaseBuilder(
            context,
            ParkMateDatabase::class.java,
            "parkmate_db"
        )   // chiamata alla callback per l'inserimento dei MockData (vedere Appendice)
            .addCallback(
                AppDatabaseCallback(
                    scope = scope,
                    provider = { database }
                )
            )
            .build()

        return database
    }
```

Inoltre è presente **`LocationEntryPoint`**, un'interfaccia che sfrutta l'annotazione @EntryPoint di Hilt per definire un "punto di accesso manuale" ai servizi di localizzazione. Infatti, questa interfaccia permette di recuperare un'istanza di **`appGeocoder`**, servizio responsabile della conversione delle coordinate geografiche in indirizzi testuali, anche in quei componenti dell'applicazione in cui non è possibile utilizzare l'iniezione automatica delle dipendenze di Hilt.

3. Dettagli di implementazione

3.1 Domain Layer

La cartella `/domain` contiene le rappresentazioni reali dei dati dell'applicazione (o, per meglio dire, le entità dell'applicazione) e le "regole" (ovvero, le interfacce e i casi d'uso) per accedervi; in altre parole, questa cartella implementa nella sua interezza la business logic di ParkMate.

```
└── domain/
    ├── models/
    ├── repository/
    ├── use_case/
    └── util/
```

Tale cartella si compone delle sottocartelle mostrate nello schema a lato, le quali contengono, rispettivamente, i modelli delle entità che compongono il database di ParkMate, implementate come data class, i casi d'uso, ovvero i wrapper delle azioni della repository destinati ad essere utilizzati dalla UI (in altre parole, l'unico modo in cui il livello dell'interfaccia utente può interagire con il database), le interfacce dei repository, cioè contratti che stabiliscono cosa l'applicazione può fare con i dati, e un insieme di funzioni di utility generiche.

3.1.1 Modelli

Modello/entità	Descrizione
`ParkingSession.kt`	Rappresenta una sessione di parcheggio dell'utente, contenente informazioni comuni a tutti e tre i possibili tipi di parcheggio (`FREE`, `HOURLY_PAID`, `FIXED_TICKET`), come la data di inizio, la posizione oppure un flag booleano che indica se sia attiva o meno, e informazioni aggiuntive specifiche del tipo di parcheggio, come il prezzo del biglietto fisso, oppure la tariffa oraria.
`Vehicle.kt`	Rappresenta il modello base per memorizzare le informazioni chiavi di un veicolo. Tali informazioni includono il nome, la tipologia del veicolo (`CAR`, `MOTORCYCLE`, `BICYCLE`, `OTHER`) e un'immagine associata, opzionale.
`ParkingEffort.kt`	Raccoglie le metriche fisiche del tracciamento pedonale (distanza, tempo, passi, dislivello, tempo impiegato) avvenuto da una specifica Parking Session alla destinazione dell'utente.
`SavedLocation.kt`	Rappresenta un luogo spesso frequentato dall'utente che quest'ultimo può registrare, contenente un nome, la posizione e l'icona scelta tra le nove disponibili.

NOTA: Nonostante i principi della Clean Architecture impongano che i modelli dichiarati all'interno del layer domain siano completamente agnostici dal data layer - in altre parole, che non contengano annotazioni o riferimenti al database - la nostra implementazione prevede che entità e modelli coincidano. Tale scelta è stata guidata da diverse motivazioni, ad esempio evitare duplicazione di codice e codice boilerplate.

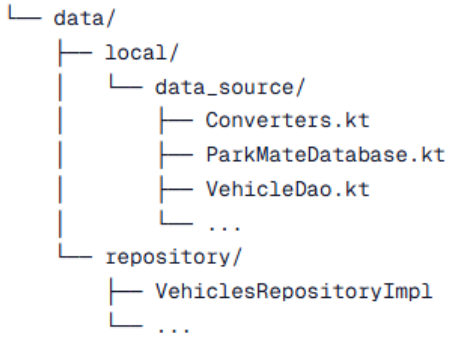
Le entità sopra riportate fanno uso di altre data class aggiuntive, come ad esempio data class `Position`, composta da latitudine, longitudine e altitudine (utilizzata nel calcolo del parking effort), oppure la data class `ParkedVehicle`, la quale associando un Vehicle alla relativa Parking Session attiva, calcola la proprietà derivata `isParked`, utile per la gestione della UI . Un altro esempio è rappresentato dalla data class data class `ParkingSessionWithVehicleInfo`, la quale mappa il risultato della query JOIN tra

ParkingSession e Vehicle e serve a recuperare in un'unica transazione di lettura sia la sessione che i dettagli essenziali del veicolo (nome e tipo), evitando di appesantire l'applicazione con multiple query individuali per ciascuna sessione.

Infine, la data class `ParkingSession` presenta due metodi atti a centralizzare due operazioni relative alle sessioni di parcheggio, quali il calcolo del costo finale (differenziato sulla base del tipo di parcheggio), per opera della procedura `calculateFinalCost()` e la raccolta dei dati necessari per l'invio delle notifiche del parcheggio in corso, grazie alla funzione `getOngoingReminderMessage()`.

3.2 Data Layer

Il package **`/data`** si occupa di interagire concretamente con le sorgenti dati (nella nostra implementazione, il database locale Room). Infatti, in questa cartella sono dichiarati:



- i Data Access Object (DAO) di Room, interfacce contenenti l'insieme di operazioni CRUD (@Query, @Update, @Delete, @Insert) per interagire direttamente con la tabella del database
- il file **`Converters.kt`**, il quale contiene le dichiarazioni di un insieme di funzioni il cui scopo è quello di convertire tipi complessi che SQLite non supporta nativamente (come ad esempio VehicleType o ParkingType)
- la classe astratta che estende RoomDatabase, `ParkMateDatabase`
- le implementazioni concrete delle varie repository le cui interfacce sono dichiarate nel package **`/domain/repository`**, le quali fungono da ponte tra data layer, domain layer e, indirettamente, presentation layer (che opera mediante gli use cases).

3.2.1 Creazione del database

Il database dell'applicazione è costruito tramite l'annotazione @Database, la quale definisce le configurazioni principali della struttura del database SQLite. Segue l'annotazione @TypeConverters e la dichiarazione del database come classe astratta, contenente le interfacce con le operazioni CRUD e le query per la lettura del database.

```
@Database(
    entities = [
        ParkingSession::class,
        Vehicle::class,
        ParkingEffort::class,
        SavedLocation::class,
    ],
    version = 1,
    exportSchema = false
)
@TypeConverters(Converters::class)
abstract class ParkMateDatabase : RoomDatabase(){
    abstract val parkingSessionDao: ParkingSessionDao
    abstract val vehicleDao: VehicleDao
    abstract val savedLocationDao: SavedLocationDao
    abstract val parkingEffortDao: ParkingEffortDao
}
```

3.2.2 Data Access Object

Ogni DAO sopra elencato rispetta la medesima logica di implementazione; è riportato un esempio esplicativo sotto (file **`ParkingEffortDao.kt`**):

```
@Dao
interface ParkingEffortDao {
    @Insert(onConflict = OnConflictStrategy.REPLACE)
    suspend fun insertParkingEffort(effort: ParkingEffort)

    @Query("DELETE FROM parking_effort WHERE sessionId = :sessionId")
    suspend fun deleteParkingEffortBySessionId(sessionId: Int)

    @Query(
        """
        SELECT
            e.sessionId AS effortId,
            s.latitude AS latitude,
            s.longitude AS longitude,
            e.effortScore AS effortScore,
            e.destination_latitude AS destination_latitude,
            e.destination_longitude AS destination_longitude
        FROM parking_effort AS e
        INNER JOIN ParkingSession AS s ON s.id = e.sessionId
        """
    )
    fun observeMapPoints(): Flow<List<EffortWithPosition>
    ...}
```

3.3 Presentation Layer & UI

Come accennato nel capitolo 2.2, ad ogni schermata dell'applicazione corrisponde un package denominato **`/feature_nomeSchermata`**, la quale, oltre allo Screen principale e al ViewModel, contiene i vari composable riutilizzabili (come ad esempio, modali, *cards*, finestre di dialogo…), inserite in apposite cartelle denominate **`/components`**. Come paradigma di costruzione dell'interfaccia grafica abbiamo optato per il toolkit Jetpack Compose data la sua natura dichiarativa, per quanto presenti maggiori tempi di build e dimensioni in termini di APK superiori rispetto al sistema tradizionale android views (fonte: https://developer.android.com/develop/ui/compose/migrate/compare-metrics).

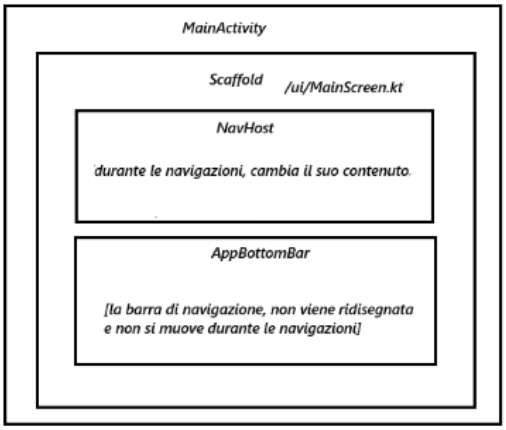
Il package **`/ui`** contiene invece i file riguardanti la gestione del theming dell'applicazione e l'implementazione della logica di navigazione. In particolare, il tema dell'applicazione è stato realizzato tramite Material Theme Builder (https://material-foundation.github.io/material-theme-builder/).
La gestione dei flussi e delle transizioni tra le schermate dell'applicazione è affidata alla libreria Navigation Compose, strutturata secondo un approccio modulare e *Type-Safe*, in particolare, tutte le rotte sono modellate come oggetti Kotlin annotati con @Serializable:

- le rotte principali sono raggruppate in un'unica interfaccia (sealed interface TopLevelRoute). La composable MainScreen coordina lo Scaffold con la AppBottomBar, evidenziando

automaticamente l'icona attiva tramite l'estensione hasRoute<T>() e gestendo il ripristino dello stato per garantire un'esperienza utente fluida al cambio di tab.

- la navigazione differita (cioè, dalle notifiche push) è gestita attraverso il parametro pendingNavigationAction. Tramite LaunchedEffect, il sistema intercetta l'azione pendente ed esegue il reindirizzamento atomico verso la destinazione corretta (`new_parking` o `home_parkings`), notificando l'avvenuta gestione via callback (`onNavigationHandled`).

Sotto sono riportati rispettivamente uno schema informale dell'architettura delle schermate e la composizione del package **`/ui`**.




```
└── ui/
    ├── components/
    │   └── AppBottomBar.kt
    ├── navigation/
    │   ├── AppNavHost.kt
    │   └── Destinations.kt
    ├── theme/
    │   ├── Color.kt
    │   ├── Theme.kt
    │   └── Type.kt
    └── MainScreen.kt
```

3.4 Background Tasks & System Services

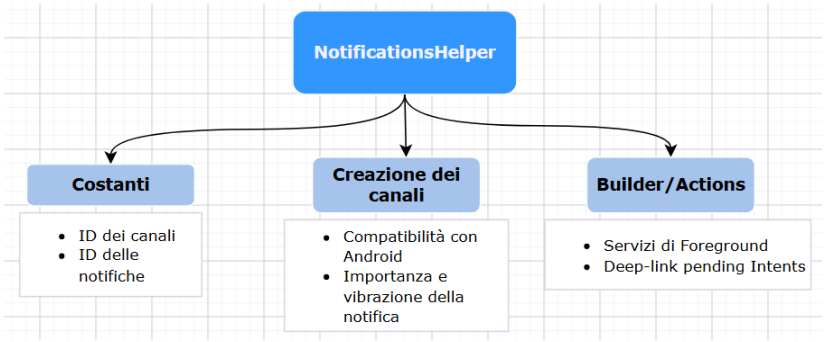
Infine, l'applicazione offre diversi servizi in foreground e in background, quali ad esempio operazioni di *Activity Recognition*, ed un sistema di notifiche, periodiche, *alerts* o innescate dall'attività fisica dell'utente - se consentito. Il funzionamento e l'implementazione di tali paradigmi è distribuito nei seguenti package:

```
└── com.example.parkmate/
    ├── notifications/
    │   └── NotificationHelper
    ├── permissions/
    ├── service/
    │   ├── activity_recognition/
    │   ├── data_providers/
    │   │   ├── location/
    │   │   │   └── LocationProvider.kt
    │   │   └── steps/
    │   │       └── StepCounterManager
    │   ├── launchers/
    │   │   └── Launchers.kt
    │   ├── receiver/
    │   │   ├── ActivityRecognitionReceiver
    │   │   └── ...
    │   └── RouteTrackingService
    ├── workManager/
    │   └── OnGoingParkingWorker.kt
    └── Background TaskManager
```

- **`/notifications`**: contiene l'intera gestione delle notifiche locali dell'applicazione
- **`/permissions`**: contiene l'oggetto RouteTrackingPermissions, che effettua controlli a runtime dei permessi concessi dall'utente necessari per il tracciamento dell'attività fisica
- **`/service`**: contiene, insieme al `Service()` necessario per la gestione dell'*Activity Recognition* in entrambe le sue forme (prompting inizio di sessione di parcheggio e inizio calcolo parking effort), l'insieme delle classi `BroadcastReceiver()`, i provider dei e le classi `@Singleton` contenenti i singoli metodi di monitoring e creazione degli intent per le due activity recognition.
- **`/workManager`**: contiene la classe OngoingParkingWorker la quale, estendendo `CoroutineWorker()`, si occupa di inviare le notifiche relative ai parcheggi in corso (non gli alert per la scadenza dei biglietti a tariffa fissa).

3.4.1 NotificationsHelper

L'oggetto **`NotificationsHelper`** funge da componente helper centralizzato per la configurazione, creazione e gestione del ciclo di vita delle notifiche locali nell'applicazione.



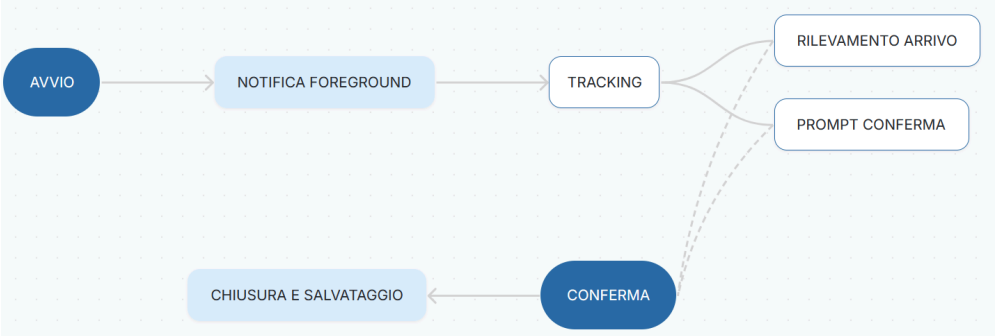
Come illustrato nello schema informale sopra riportato, le funzionalità principali di questo componente possono essere riassunte nei seguenti punti:

1. Raccoglie le costanti globali per gli ID dei canali (es. `PARKING_EXPIRY_CHANNEL_ID`, …) e gli ID univoci delle notifiche (es. `CONFIRMATION_NOTIFICATION_ID`, …). Si è optato per l'utilizzo di ID numerici distinti al fine di prevenire conflitti nel sistema Android e di favorire l'aggiornamento o la cancellazione delle singole notifiche. Vi sono cinque canali di notifiche, ovvero, il canale dedicato al tracciamento del percorso (necessario per il calcolo del parking effort), il canale per la notifica sulla transazione (dal veicolo a piedi, per suggerire all'utente di registrare o gestire le proprie sessioni di parcheggio), il canale che invia notifiche alert in merito alla scadenza prossima di biglietti a tariffa fissa e il canale atto ad aggiornare l'utente sullo stato dei propri parcheggi (entrambi attivabili/disattivabili al momento del salvataggio di una nuova sessione) ed un canale per le notifiche di errore.
2. Effettua l'inizializzazione dei canali tramite l'esposizione di metodi dedicati (es. `createNotificationChannel()`) per la registrazione dei NotificationChannel sul sistema Android, obbligatori da API 26 in poi. Ogni canale definisce un proprio livello di priorità e comportamento (es. `IMPORTANCE_HIGH` con vibrazione per scadenze/errori, oppure `IMPORTANCE_LOW` senza badge per i servizi silenti in background).
3. Costruisce gli oggetti Notification collegati ai servizi di background (**`RouteTrackingService`**) attraverso l'utilizzo di apposite funzioni builder (es. `buildTrackingNotification()`). Le notifiche sono rese interattive tramite PendingIntent immutabili (`FLAG_IMMUTABLE`) che consentono all'utente di eseguire azioni rapide direttamente dalla barra delle notifiche (es. interrompere o annullare il tracciamento) senza dover aprire manualmente l'interfaccia dell'app.
4. Nel metodo `buildVehicleTransitionNotification()`, l'helper gestisce il cambio di stato rilevato dall'Activity Recognition (da veicolo a pedone). In base alla presenza di sessioni attive (`hasActiveSessions`), imposta dinamicamente l'Intent per reindirizzare l'utente direttamente alla schermata corretta (`home_parkings` o `new_parking`), collegandosi con la gestione del parametro pendingNavigationAction visto nel componente di navigazione principale.

3.4.2 Route Tracking Service

Il sistema `RouteTrackingService` è un'infrastruttura completa (*Foreground Service*) per il tracciamento in background dei percorsi pedonali in seguito del parcheggio di un veicolo. Per esaminare quanto più fedelmente lo sforzo compiuto dall'utente durante il tragitto, si è scelto di optare per la combinazione di diversi funzioni di tracciamento : la rilevazione della posizione GPS, il conteggio dei passi, il riconoscimento dell'attività motoria (*Activity Recognition*) e la gestione proattiva delle notifiche foreground/background. Inoltre il sistema si compone di numerose componenti distinte, ognuna delle quali si occupa di una task ben precisa e concisa.

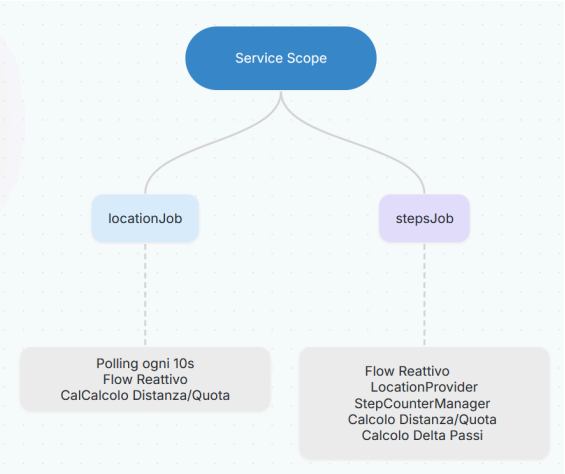
3.4.2.1 Ciclo di vita



5. **Avvio**: Con `ACTION_START` invocato da `BackgroundTaskManager` il servizio si avvia come Foreground Service (garantendo la conformità alle API 34+ per i tipi LOCATION e HEALTH), valida i permessi ed esegue il boot dei thread di monitoraggio.
6. **Monitoraggio attivo**: prevede l'azione coordinata di:
    - GPS: Esegue un polling della posizione ogni 10 secondi tramite `LocationProvider`, accumulando la distanza percorsa.
    - Passi: Registra i passi tramite `StepCounterManager`, calcolando la variazione rispetto al valore letto all'avvio.
    - Tempo: un timer tiene traccia della durante complessiva del servizio e di conseguenza del percorso dell'utente
    - Altitudine : viene calcolata la differenza di altitudine tra la posizione del parcheggio (inizio tracciamento) e l'arrivo a destinazione (fine tracciamento).
    - Rilevamento Arrivo: Quando l'Activity Recognition rileva lo stato di fermo (STILL), viene mostrata una notifica ad alta priorità per chiedere conferma all'utente (*"Sei arrivato a destinazione?"*).
    - Chiusura e Salvataggio (`ACTION_STOP_AND_SAVE`): Alla conferma o allo stop manuale, il servizio calcola durata totale, distanza, passi e dislivello positivo, persistendo i dati tramite `CompleteRouteTrackingUseCase`. Segue il cleanup delle risorse, la cancellazione delle notifiche e l'arresto del servizio.

3.4.2.2 Struttura

All'interno di **`RouteTrackingService`** agiscono in maniera coordinata diversi attori: `BackgroundTaskManager`, l'entry-point centrale per l'invio di comandi al servizio; `LocationProvider`, provider indipendente e dedicato alla geolocalizzazione ad alta precisione; `StepCounterManager`, gestore del sensore hardware del contapassi; `RouteTrackingActivityRecognitionManager`, gestisce il monitoraggio della stato di transizione motoria (da camminata a fermo) durante il tracciamento, per rintracciare l'arrivo a destinazione `ActivityRecognitionReceiver`, `RouteTrackingActionReceiver` intercettano eventi dal SO o le azioni dell'utente sulle notifiche, instradandole ai relativi manager.



3.4.2.2 Scelte implementative e controllo errori

Per ottenere un'approssimazione quanto più vicina alla distanza reale percorsa dall'utente, si è scelto di rilevare la posizione dell'utente ogni 10 secondi e di calcolare per ogni nuova posizione, la distanza accumulata rispetto alla posizione precedente. Questo metodo consente di non consumare eccessivamente la batteria. Inoltre nel rilevare la posizione si fanno anche uso di una cache interna (per lo storage rapido dei dati) e di un sistema di sicurezza basato su fallback : infatti si tenta fino a 3 volte l'acquisizione della posizione e se quest'ultima non viene trovata, si esegue un fallback immediato su `lastLocation`. Altre scelte implementative degne di nota vertono sull'uso di un *serviceScope* basato su `limitedParallelism(1)` per evitare race condition nella mutazione dei dati di tracciamento e `SupervisorJob` per isolare eventuali fallimenti dei singoli coroutine job.

3.5 Impostazioni

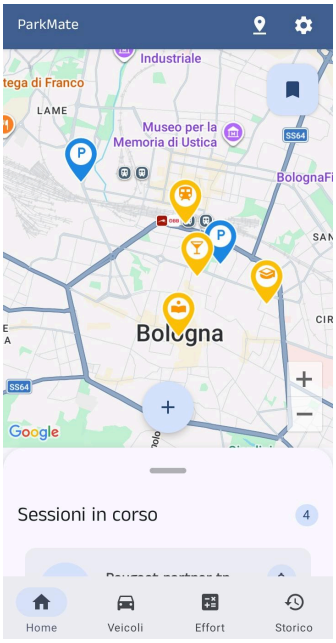
Il modulo **`/preferences`** gestisce la persistenza delle impostazioni dell'applicazione (come la geolocalizzazione automatica o il rilevamente delle attività per la schermata home), seguendo la medesima struttura Clean Architecture del resto del progetto (**`/data`**, **`/domain`**, **`/di`**). In particolare, nello strato dati, la classe `SettingsDataStore` incapsula la libreria Jetpack DataStore (Preferences), che garantisce la consistenza dei dati senza rischio di blocchi dell'interfaccia grafica.

Nel momento in cui l'utente abilita il rilevamento delle attività e la geolocalizzazione, tramite la funzione `observeActivityRecognitionSetting()` che opera nel file HomeViewModel viene abilitato il monitoraggio dell'attività a seguito della chiamata all'apposito manager (`vehicleActivityRecognitionManager.startMonitoring()`). Quest'ultimo rileva le transazioni in uscita da uno stato sul veicolo o in bicicletta e le transazioni (`IN_VEHICLE`, `ON_BICYCLE`) in entrata nello stato camminata/fermo (`WALKING`, `STILL`).

4. Interfaccia utente

4.1 Schermata Home (*prime tre foto dalla sinistra*)

La schermata principale dell'applicazione si compone di una mappa di Google Maps che offre la possibilità di visualizzare la posizione delle sessioni di parcheggio attive e, se selezionate, anche le Saved Locations dell'utente, caratterizzate da marker di colori diversi e con icone che cambiano sulla base della scelta dell'utente. Nell'area inferiore dello schermo è inoltre presente una *Bottom Sheet* sempre parzialmente visibile (o, estendibile) che riporta un elenco delle sessioni di parcheggio attive tramite cards. Il click su una di queste ultime (o su uno dei marker sulla mappa), apre una modale di dettaglio sulla singola sessione, incluso il tempo trascorso dall'inizio di quest'ultima o, nel caso di parcheggi a tariffa fissa, il tempo rimanente prima della sanzione. Infine, dalla barra di navigazione in alto è possibile raggiungere la schermata delle Saved Locations e le impostazioni.









4.2 Schermata Veicoli (*ultima foto a destra*)

4.2.1 Gestione dei veicoli

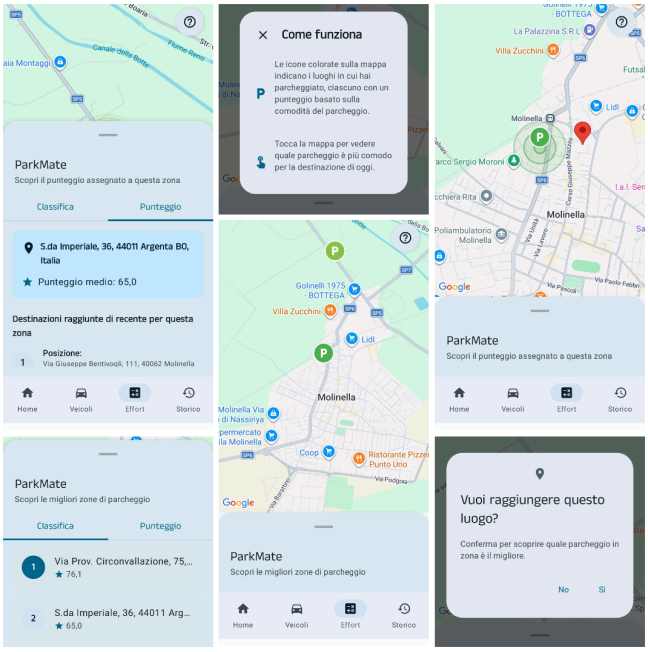
All'interno di tale sezione l'utente è in grado di visualizzare e gestire in maniera rapida, i propri veicoli registrati. La schermata si articola in 3 macro-sezioni : una barra superiore, utile per ricercare rapidamente nei veicoli, un sistema di filtri combinati e infine un'elegante lista che mostra le schede dei veicoli.

Ogni scheda dà la possibilità di visualizzare ed eventualmente modificare le informazioni principali del proprio veicolo: nome e immagine, stato di sosta (tra disponibile e parcheggiato) e topologia associata. Per registrare un nuovo veicolo, invece, basterà cliccare sul pulsante presente in basso a destra dello schermo, per compilare un Form dedicato. Nel caso in cui si desideri eliminare un mezzo registrato, sarà sufficiente scorrere verso destra con il dito, sulla relativa scheda.

4.3 Schermata Parking Effort

4.3.1 Mappa interattiva

Tale schermata rappresenta il punto di accesso e il cuore della visualizzazione del Parking Effort all'interno di ParkMate. Dopo la valutazione del tragitto da un parcheggio alla relativa destinazione, un marcatore colorato evidenzia il punto in cui si è parcheggiato e lo colora in base ai dati raccolti. Una volta che si è selezionata un'area di parcheggio, un'animazione a impulso mostra la relativa area di interesse che può variare se vengono registrate nuove aree : infatti se un nuovo luogo viene registrato in prossimità di uno già esistente, le due aree vengono unificate.



4.3.2 Bottom Sheet

Nella parte bassa della schermata è presenta una *Bottom Sheet*, dove vengono visualizzate i dati notevoli sulle aree di parcheggio. Tali informazioni includono una classifica delle Parking Zones registrate, in base ai rispettivi punteggi e un elenco delle destinazioni raggiunte di recente da ogni zona selezionata. Nel caso in cui non via sia accesso alla connessione di rete sarà comunque possibile identificare i luoghi di interesse, tramite le corrispondenti coordinate geografiche.

4.3.3 Funzionalità aggiuntive

Sono inoltre presenti funzionalità aggiuntive, come la possibilità di riconoscere direttamente sulla mappa quali destinazioni sono state raggiunte da un dato luogo di parcheggio, semplicemente cliccando sopra il corrispondente marcatore ; la disponibilità di interrogare una finestra di aiuto nel caso in cui non si sappia come interagire con la schermata e infine la possibilità di sapere con un semplice tocco quale parcheggio si adatta maggiormente a una data destinazione : basterà infatti cliccare sul luogo che si vuole raggiungere, per essere indirizzati al migliore parcheggio nelle vicinanze.

4.4 Schermata Storico

4.4.1 Tab Lista

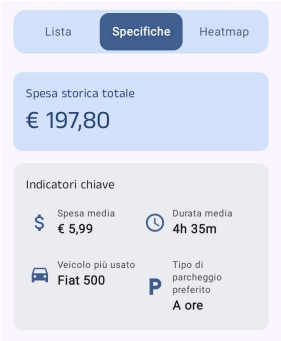
La schermata dello storico si compone di tre *tab* principali, Lista, Specifiche e Heatmap. La prima citata mostra un semplice elenco di cards contenenti le sessioni di parcheggio registrate e concluse dall'utente, con possibilità di filtrarle sulle base di diversi criteri, quali: intervallo di tempo, nome del veicolo e tipo di parcheggio. Nel momento in cui si clicca su una delle cards associate alla sessione, si apre una finestra di dettaglio analoga a quella presentata nella schermata Home (*foto a lato*)



4.4.2 Tab Specifiche (Analytics)

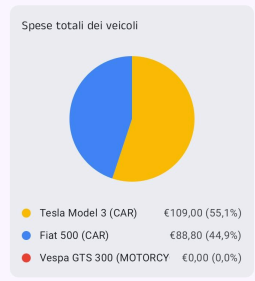
La seconda tab presente nella schermata dello storico è quelle delle specifiche (o analytics), la quale mostra l'insieme dei dati passati delle sessioni di parcheggio, raggruppati secondo diversi criteri e renderizzati tramite l'uso di apposite card. In questa schermata, l'utente può visualizzare:

- la spesa storica totale
- un insieme di indicatori chiave (KPI) che raccolgono la spesa media mensile, il veicolo più usato, la durata media del parcheggio e il tipo di parcheggio preferito
- l'importo speso per mese mostrato tramite diagramma cartesiano VICO o la percentuale che ciascuna delle due categorie di parcheggio (a ore e a tariffa fissa) occupa della spesa totale
- il numero di sessioni di parcheggio per mese tramite diagramma cartesiano VICO
- le spese totali raggruppate per veicolo in un grafico a torta



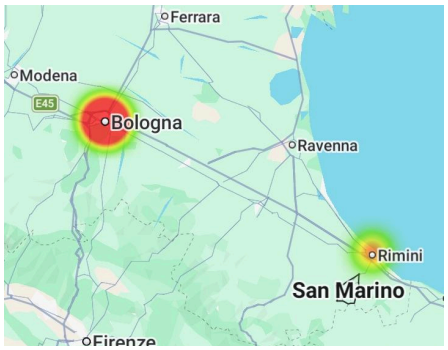


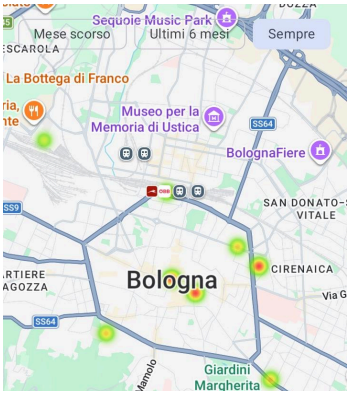




4.4.3 Tab Heatmap

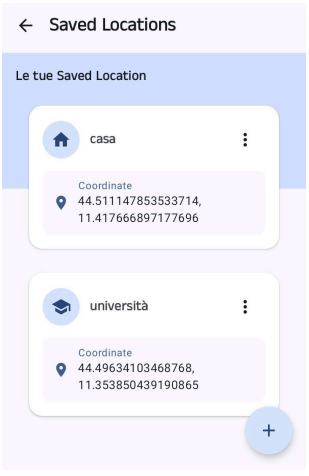
La terza ed ultima tab mostra una heatmap che consente all'utente di avere un colpo d'occhio delle zone più frequentate; i punti che presentano più sessioni di parcheggio sono di colore rosso, viceversa, laddove è presente una minore attività viene utilizzato il colore verde. Inoltre, è possibile osservare l'andamento delle sessioni di parcheggio attraverso il tempo, filtrandole per "ultimo mese", "ultimi sei mesi" e "sempre".





4.5 Saved Locations

Nella schermata delle Saved Locations è possibile visualizzare, modificare o eliminare le posizioni salvate, caratterizzate da un nome, coordinate geografiche ed un'icona personalizzabile a scelta tra quelle disponibili, ad ognuna delle quali corrisponde un determinato marker visualizzabile nella mappa principale della Home Screen.





Appendice

Per garantire la dimostrazione completa delle funzionalità dell'applicazione (storico sessioni, filtri temporali e statistiche) senza alterare il codice destinato alla produzione, è stato implementato un meccanismo di *Database Seeding* condizionale. Nello specifico, nel file **`build.gradle.kts`** (modulo app) è stata definita la flag booleana `ENABLE_MOCK_DATA`, valorizzata su *true* per le build di debug e su *false* per quelle di release; l'inserimento dei dati è gestito tramite la classe `AppDatabaseCallback`, più nello specifico, nel suo metodo `onCreate()` si verifica lo stato di `BuildConfig.ENABLE_MOCK_DATA`, dove, nel caso in cui quest'ultimo sia impostato a true, la callback inizializza una coroutine per il popolamento asincrono delle entità principali (Vehicle e ParkingSession). Il dataset di test è contenuto nel file **`MockData.kt`**, realizzato tramite intelligenza artificiale.

NOTA: Al momento della consegna del progetto, buildConfig è impostato a true col fine di facilitare, per l'appunto, il testing della schermata storico. Se non si desidera popolare il database con i dati mock al momento dell'installazione dell'app, è sufficiente navigare nel file **`build.gradle.kts`** (modulo app), in particolare sotto la voce android, in buildTypes, sino ad arrivare al blocco `debug {buildConfigField("Boolean", "ENABLE_MOCK_DATA", "true")}` ed impostare il valore a false.